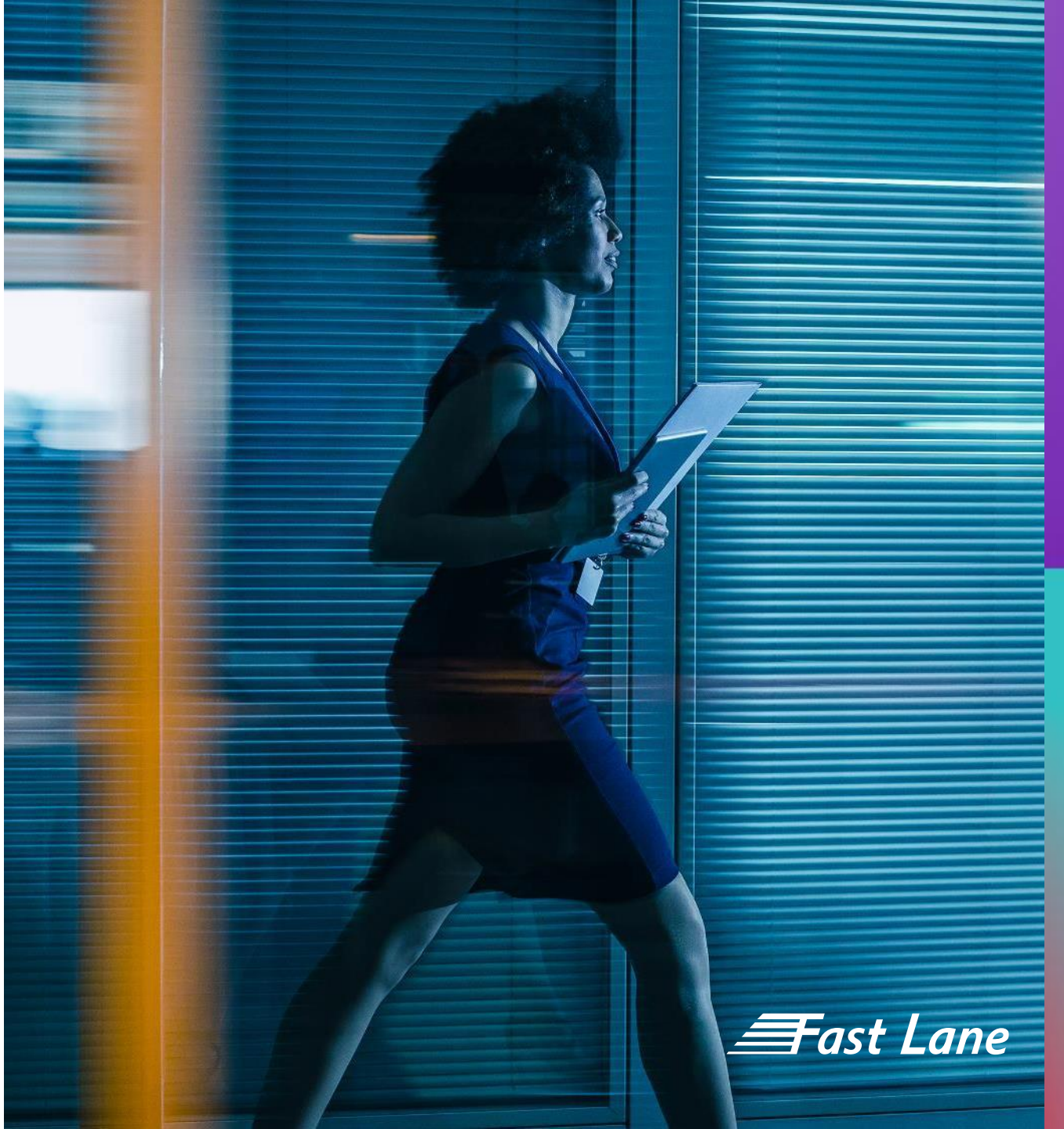
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 8

Durable Execution, Workflows & Sagas

Code that lets a long-running process survive restarts

Event-Driven Distributed Apps on Azure: Aspire, Dapr & Durable Execution



01

Durable Execution: Replay & Determinism

Event history, replay and the determinism rules

02

Durable Task Scheduler: Managed Backend

Task hubs, SKUs, the emulator and managed identity

03

Durable Functions: Orchestrations & Entities

Isolated worker, entities and the azureManaged provider

04

Durable Timers: External Events & Approvals

Delivery deadlines, approvals and RaiseEventAsync

05

Durable Task SDK: KEDA-Scaled Workers

Generic host workers that scale on the backlog

06

Dapr Workflow: Portable Orchestrations

Dapr.Workflow, Dapr 1.18 and AKS hosting

07

Saga Orchestration: Compensating Activities

Compensations in reverse, step roles and coordination

Durable execution, workflows and sagas

Code that lets a long-running process survive restarts

- **Replay and determinism**: the event history that lets an orchestration resume where it stopped
- **Backends and hosts**: the Durable Task Scheduler, Durable Functions, KEDA-scaled workers and Dapr Workflow
- **The booking saga**: delivery deadlines, customer approvals and compensations that undo finished steps



Overview

The path through durable execution

- Start with **replay and determinism**, the rules every later topic depends on
- Keep orchestration state in the **Durable Task Scheduler** and run it with Durable Functions
- Wait for deadlines and approvals with **durable timers** and external events
- Scale SDK workers with **KEDA**, or pick **Dapr Workflow** as the portable option
- Combine the pieces into the **booking saga** with compensating activities



01

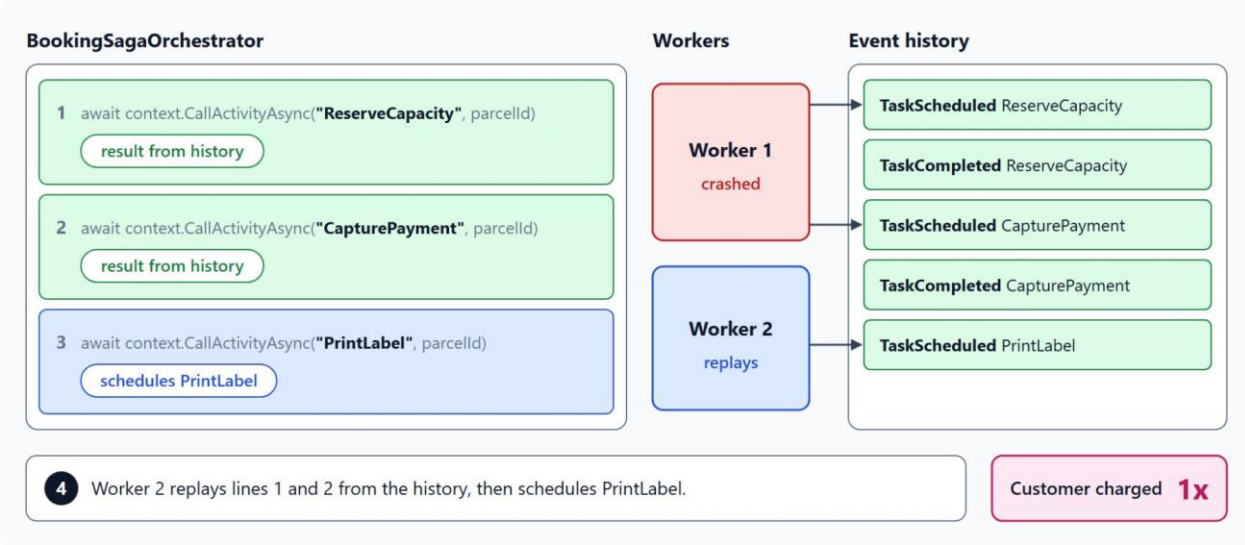
Durable Execution: Replay & Determinism

Event history, replay and the determinism rules

How replay rebuilds state

The event history lets a second worker finish the booking

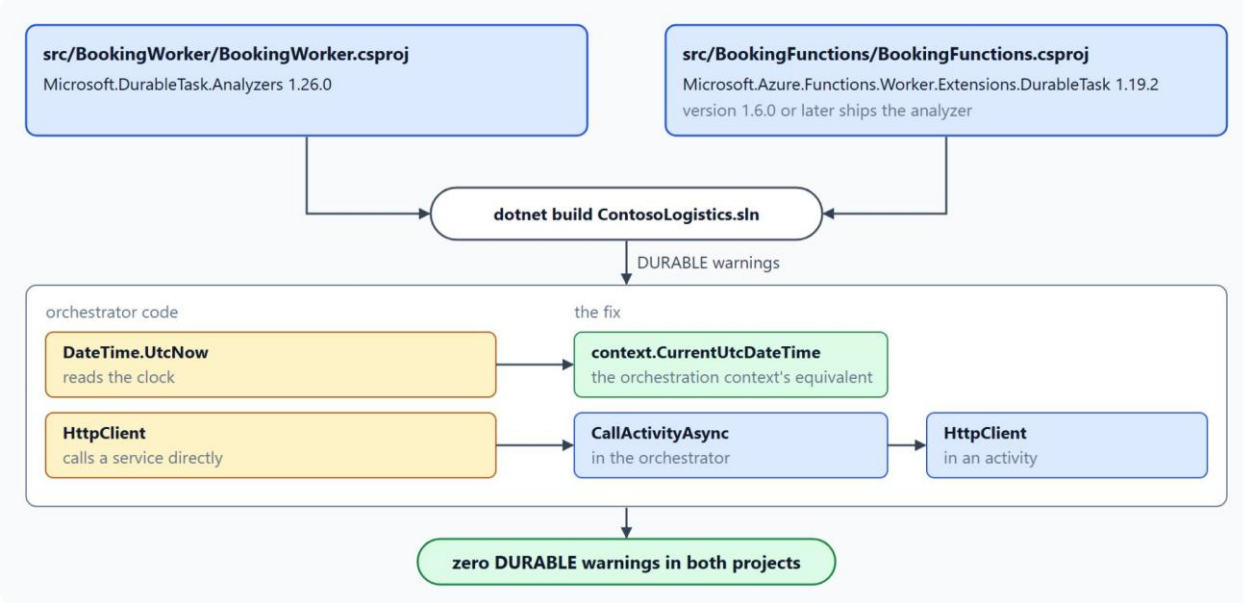
- Every scheduled activity, completed activity, timer and external event lands in the instance's **event history**
- A worker runs the orchestrator from the first line, and each `await` with a recorded result returns it at once
- After a crash, the next worker replays the history and schedules only `PrintLabel`, so the customer pays once
- Local variables come back from the history; the code never writes them to a database
- Between checkpoints the orchestrator leaves memory, so a long wait holds no worker



Determinism rules

Context APIs and activities replace calls that change on replay

- Every replay must make the same decisions, so the clock, random values and network calls stay out
- `context.CurrentUtcDateTime`, `context.NewGuid()` and `context.CreateTimer` replace the clock, GUIDs and delays
- **Activities** hold the side effects: `HttpClient`, storage clients and `Random`
- Activities run at least once, so each one must be idempotent
- The Roslyn analyzer reports DURABLE0001 to DURABLE0011 before you merge



02

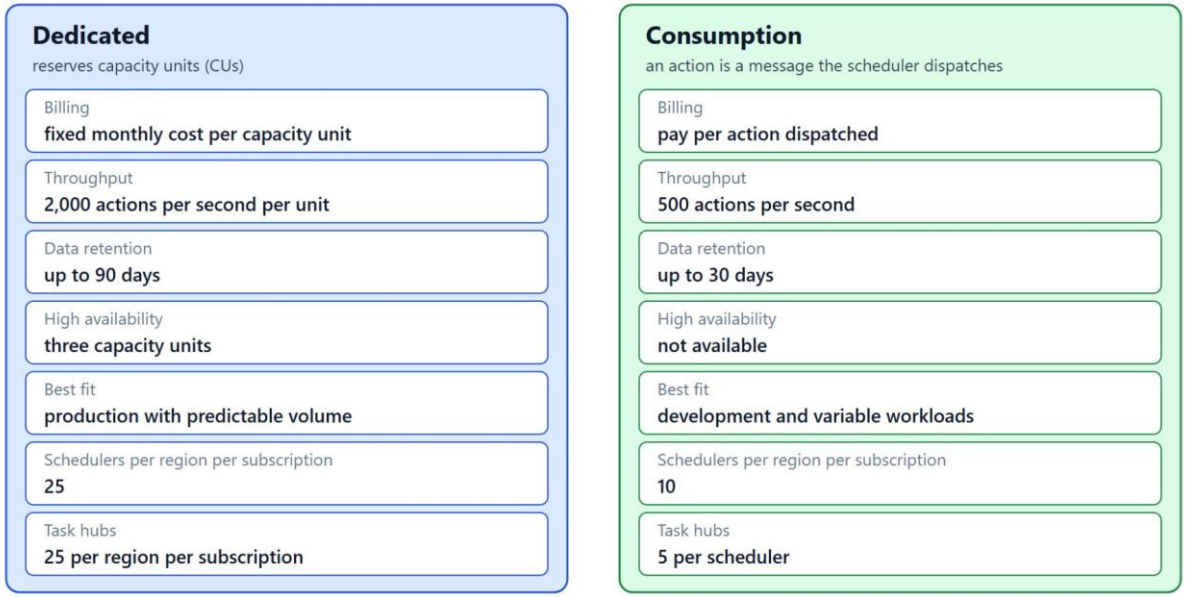
Durable Task Scheduler: Managed Backend

Task hubs, SKUs, the emulator and managed identity

Schedulers, task hubs and SKUs

Dedicated reserves capacity, Consumption bills per action

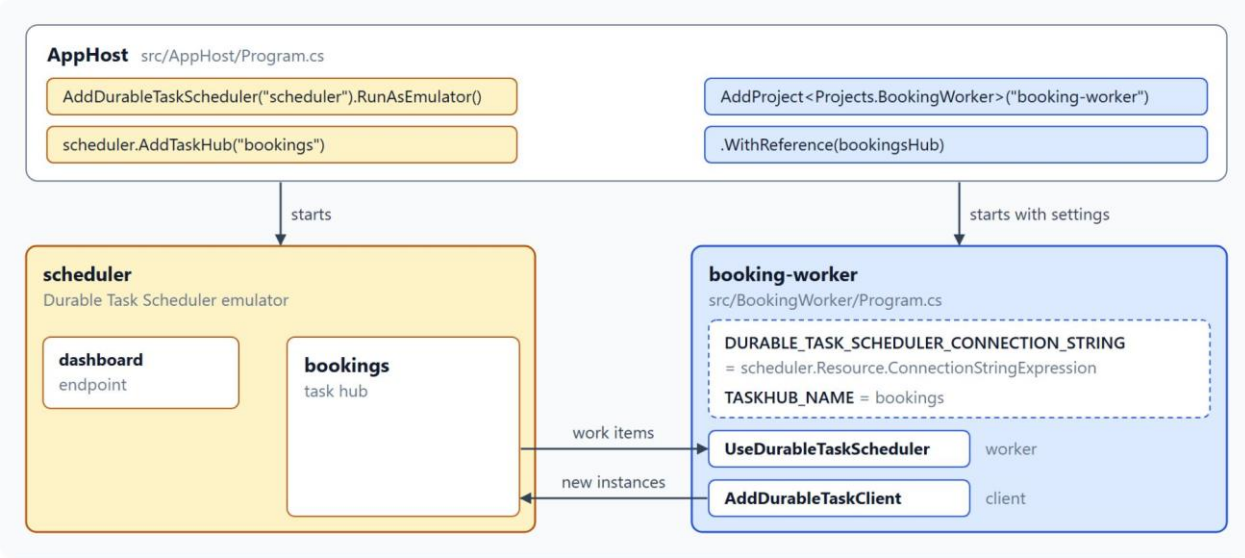
- A **scheduler** is the Azure resource; a task hub inside it holds the state of a set of orchestrations
- Each app connects to one task hub, created with the `durabletask` CLI extension
- Dedicated reserves capacity units at a fixed monthly cost, 2,000 actions per second each
- Consumption bills per dispatched action, at 500 actions per second
- Inputs and outputs are limited to 1 MB, instance IDs to 100 printable ASCII characters



Emulator and connection strings

Local runs on the emulator, Azure apps use managed identity

- The **emulator** runs the same protocol in Docker: port 8080 for the endpoint, 8082 for the dashboard
- Under Aspire, `RunAsEmulator()` starts it and passes `ConnectionStringExpression` to the app
- Local: `Endpoint=http://localhost:8080;Authentication=None`, with the task hub from `TASKHUB_NAME`
- Azure: `Authentication=ManagedIdentity` with the client ID of a user-assigned identity
- Grant Durable Task Data Contributor at the task hub scope



03

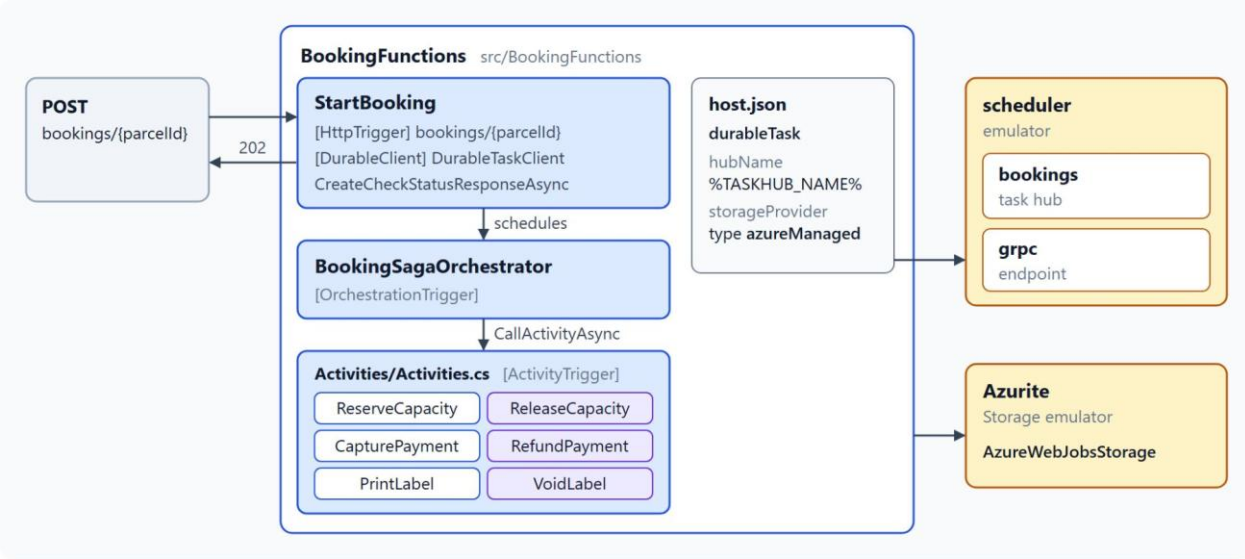
Durable Functions: Orchestrations & Entities

Isolated worker, entities and the azureManaged provider

Orchestrations in the isolated worker

Orchestrator, activities and client on the azureManaged provider

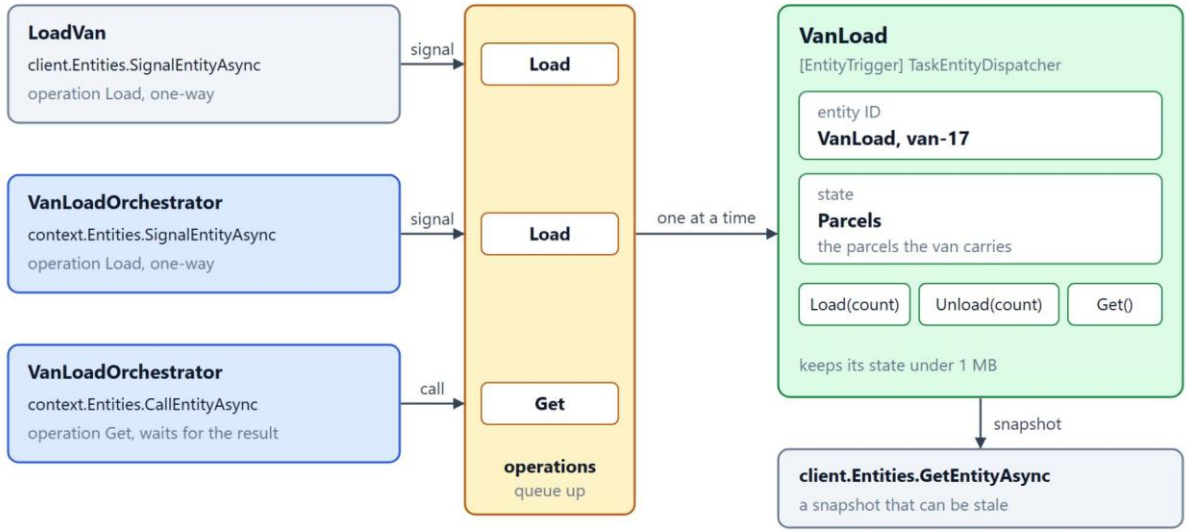
- The in-process model ends support on 10 November 2026, so use the **.NET isolated worker**
- `[OrchestrationTrigger]` runs the orchestrator, `[ActivityTrigger]` the activities that do the I/O
- The starter schedules through `[DurableClient]` and returns `CreateCheckStatusResponseAsync`
- `host.json` selects the `azureManaged` provider and names the connection string setting
- `hubName` must match the task hub that holds the instances



Durable entities for van load

One entity per van handles its operations one at a time

- An **entity** holds a small piece of state and accepts operations from many callers
- Contoso models each van as one `VanLoad` entity keyed by the van ID, such as `van-17`
- A signal is one-way; a call from an orchestrator waits for the result
- Concurrent signals queue up, so no two operations update the state at once
- A read returns a snapshot that can be stale; keep each entity under 1 MB



04

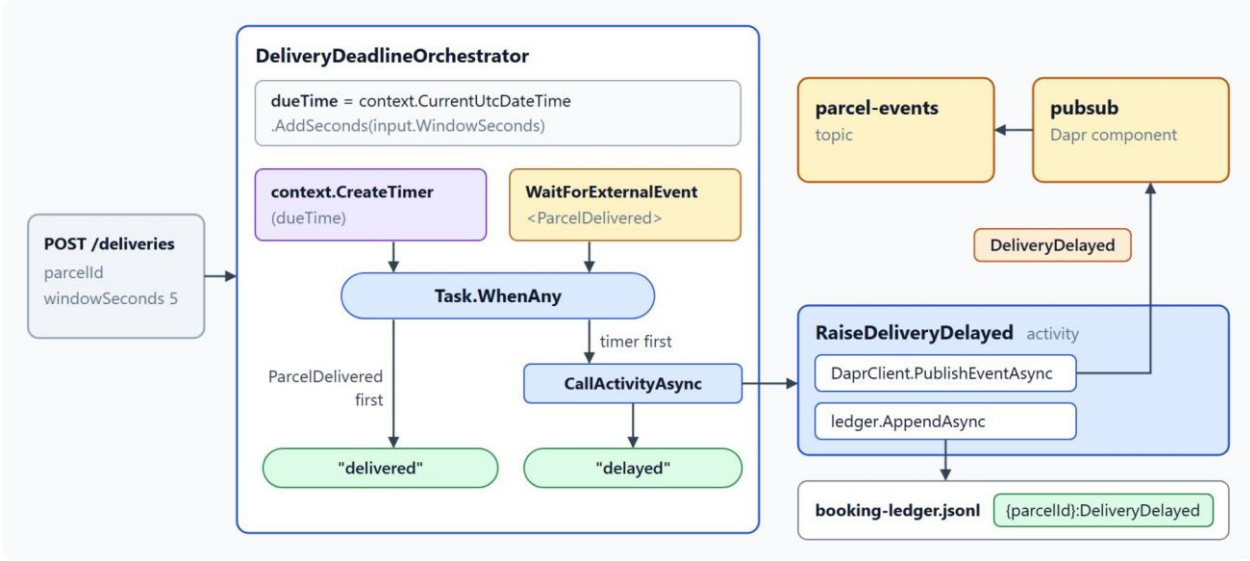
Durable Timers: External Events & Approvals

Delivery deadlines, approvals and RaiseEventAsync

Durable timers and the delivery deadline

A durable timer races the ParcelDelivered event

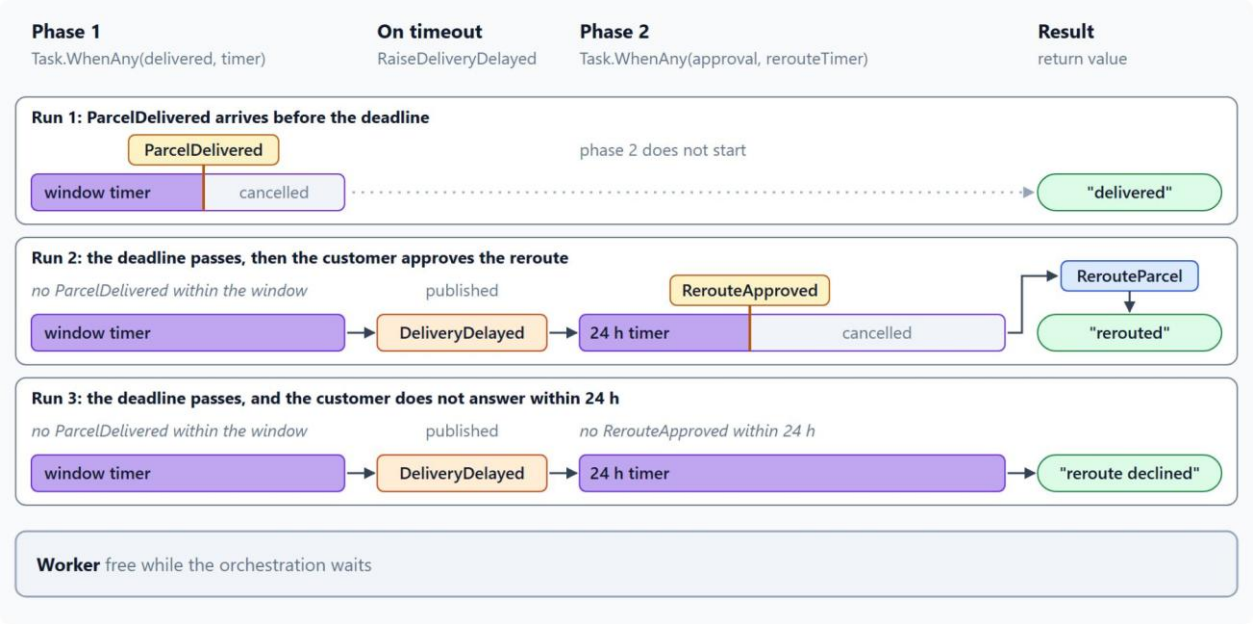
- `CreateTimer` completes at a future time; the timer is stored with the instance and survives a restart
- Compute the due time from `context.CurrentUtcDateTime`, never `DateTime.UtcNow`
- `WaitForExternalEvent<ParcelDelivered>` and the timer race in `Task.WhenAny`
- When the timer wins, `RaiseDeliveryDelayed` publishes `DeliveryDelayed` to `parcel-events`
- Cancel the timer that loses, because an outstanding timer keeps the orchestration open



Reroute approval with external events

The approval races a 24-hour timer, RaiseEventAsync delivers it

- An **external event** is a named message, and the name must match on both sides
- Early events are buffered; events for unknown instances are discarded
- Delivery is at least once, so put a unique ID in the payload
- `RerouteApproved` races a 24-hour timer; a timeout returns reroute declined
- `RaiseEventAsync` or the HTTP `raiseEvent` API delivers the approval behind a function key



05

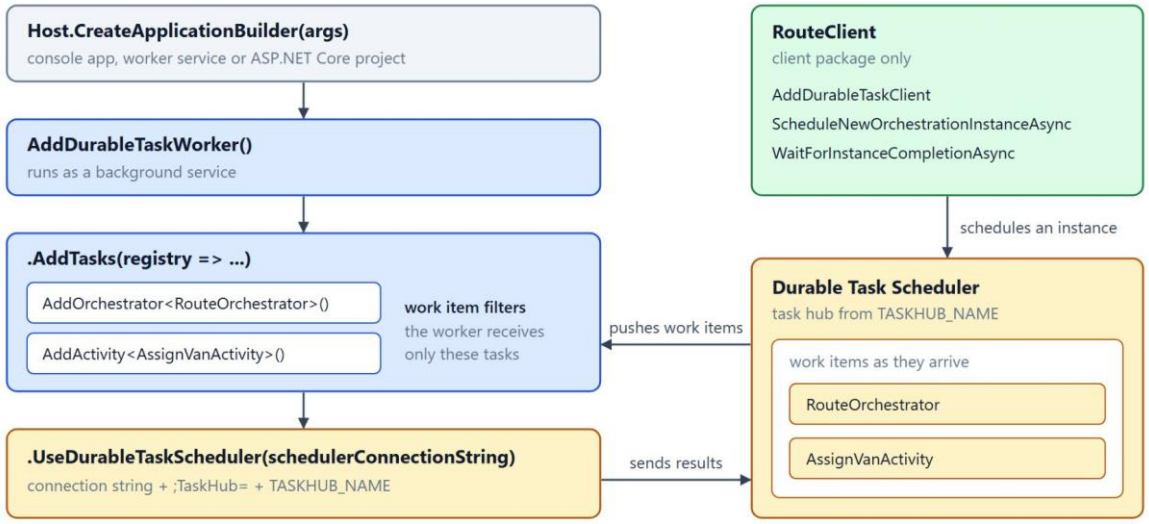
Durable Task SDK: KEDA-Scaled Workers

Generic host workers that scale on the backlog

Worker and client packages

Orchestrations and activities on a generic host

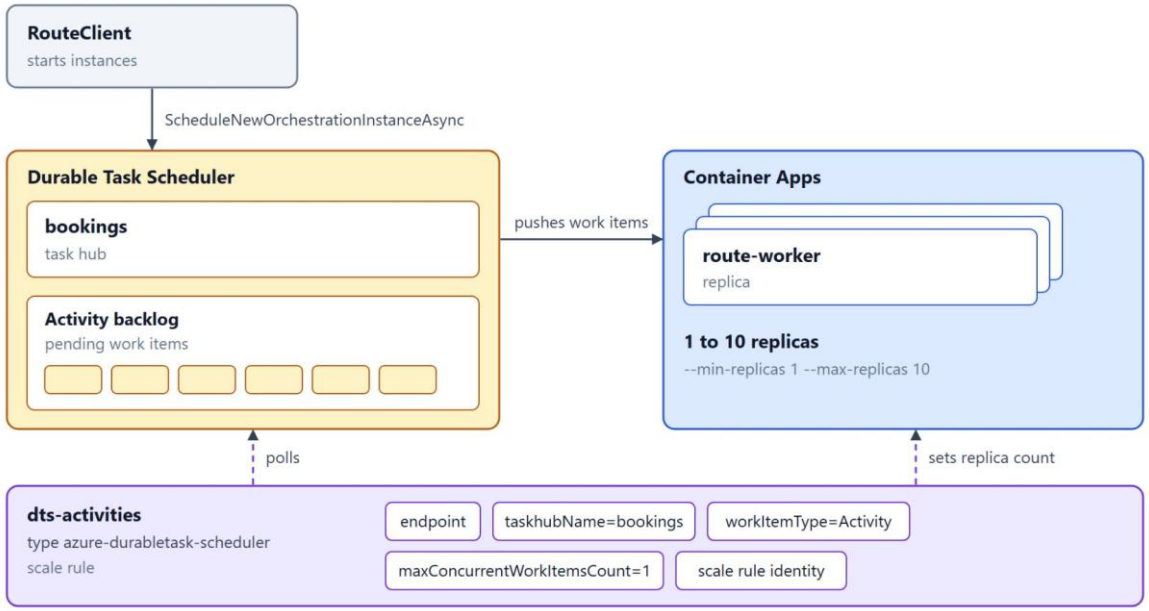
- The **AzureManaged** worker and client packages add `UseDurableTaskScheduler` to the builder
- The worker registers `RouteOrchestrator` and `AssignVanActivity` during host setup
- It runs as a background service in a console app, worker service or ASP.NET Core project
- The scheduler pushes work items, and filters send each worker only the tasks it hosts
- A service that starts instances installs only the client package



KEDA scale rule for workers

Container Apps adds replicas as the activity backlog grows

- A custom rule of type `azure-durabletask-scheduler` polls the scheduler for pending work items
- It sets `endpoint`, `taskhubName`, `workItemType` and `maxConcurrentWorkItemsCount`, plus a managed identity
- A lower `maxConcurrentWorkItemsCount` adds replicas sooner; start at 1 for CPU-bound work
- `minReplicas` 0 scales to zero, at the cost of a cold start
- A CLI update replaces the rule, so add the orchestration rule with YAML or Bicep



06

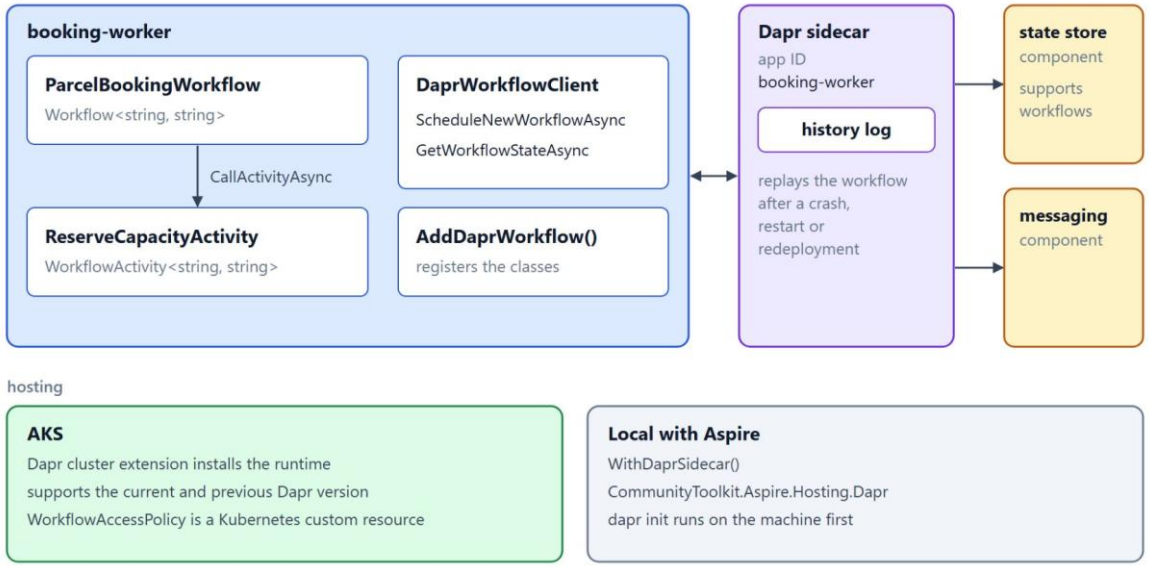
Dapr Workflow: Portable Orchestrations

Dapr.Workflow, Dapr 1.18 and AKS hosting

Workflows and activities in Dapr

A sidecar next to each service keeps the workflow history

- A workflow derives from `Workflow<TInput, TOutput>`, an activity from `WorkflowActivity<TInput, TOutput>`
- `DaprWorkflowClient` starts with `ScheduleNewWorkflowAsync` and reads `GetWorkflowStateAsync`
- Dapr replays the code against its history log, so the determinism rules still apply
- Dapr 1.18 can sign the history, and `WorkflowAccessPolicy` controls which apps invoke a workflow
- Run it on **AKS** with the Dapr extension; locally, Aspire attaches the sidecar



Dapr Workflow and Durable Task compared

Choose by hosting, portability and operations

- Both persist progress and replay the code under determinism rules after a failure
- Durable Task keeps its state in the managed **Durable Task Scheduler**
- Dapr Workflow keeps it in a state store component and runs on AKS
- Choose Durable Task on Functions or Container Apps with a managed backend
- Choose Dapr Workflow when the service already runs on AKS with Dapr

	Durable Task Functions and SDKs	**Dapr Workflow**
.NET package	Microsoft.DurableTask.Worker.AzureManaged, or the Functions extension	Dapr.Workflow
State backend	Durable Task Scheduler	a state store component that supports workflows
Start and read state	ScheduleNewOrchestrationInstanceAsync, status URLs	ScheduleNewWorkflowAsync, GetWorkflowStateAsync
External events	RaiseEventAsync	raise event through the HTTP or gRPC management API
Azure hosting	Functions, Container Apps, AKS, App Service and VMs	AKS through the Dapr extension
Upgrade model	Azure-managed backend	Dapr runtime version that the team schedules
After a failure	both replay the code against its history under determinism rules	
Choose when	**the service runs on Functions or Container Apps, and the backend is a managed Azure service**	**the service already runs on AKS with Dapr, one component model for state, messaging and workflows**

07

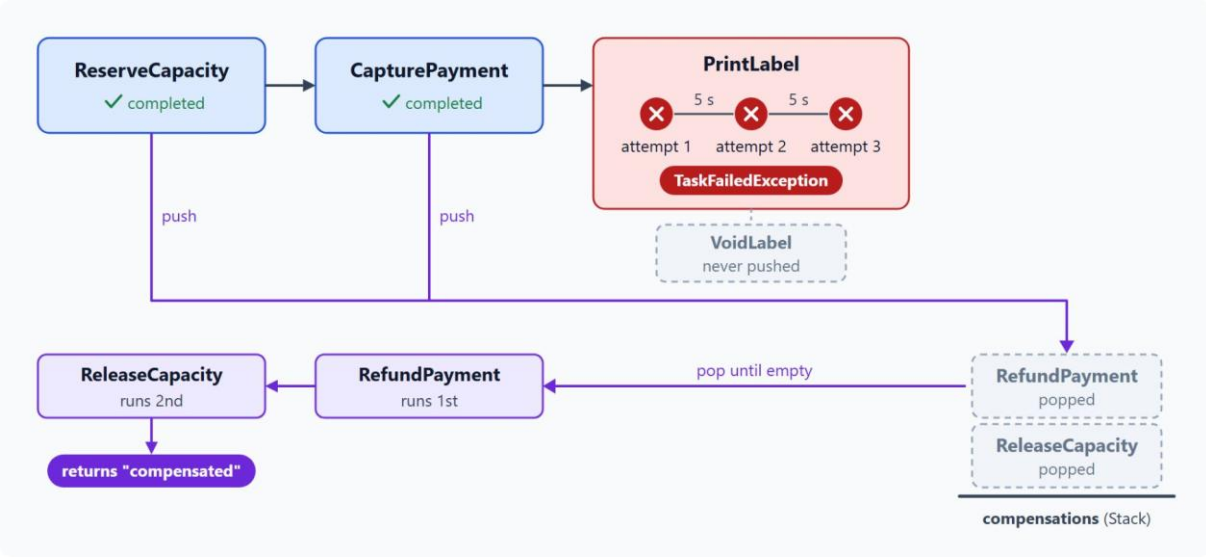
Saga Orchestration: Compensating Activities

Compensations in reverse, step roles and coordination

The booking saga and its compensations

Completed steps are undone in reverse order

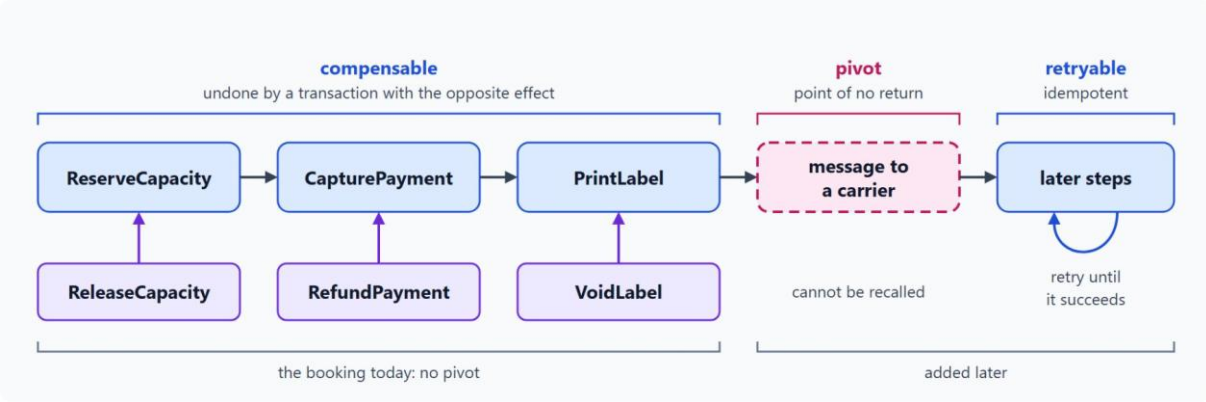
- Capacity, payment and the label each change their own database, so no single transaction covers them
- Each forward step has one compensation: `ReleaseCapacity`, `RefundPayment`, `VoidLabel`
- The orchestrator pushes a compensation onto a stack as each step completes
- A `TaskFailedException` pops the stack, so the refund runs before the capacity release
- A compensation that throws fails the orchestration, so make each one retryable



Pivot, retryable and compensable steps

The booking has no pivot today

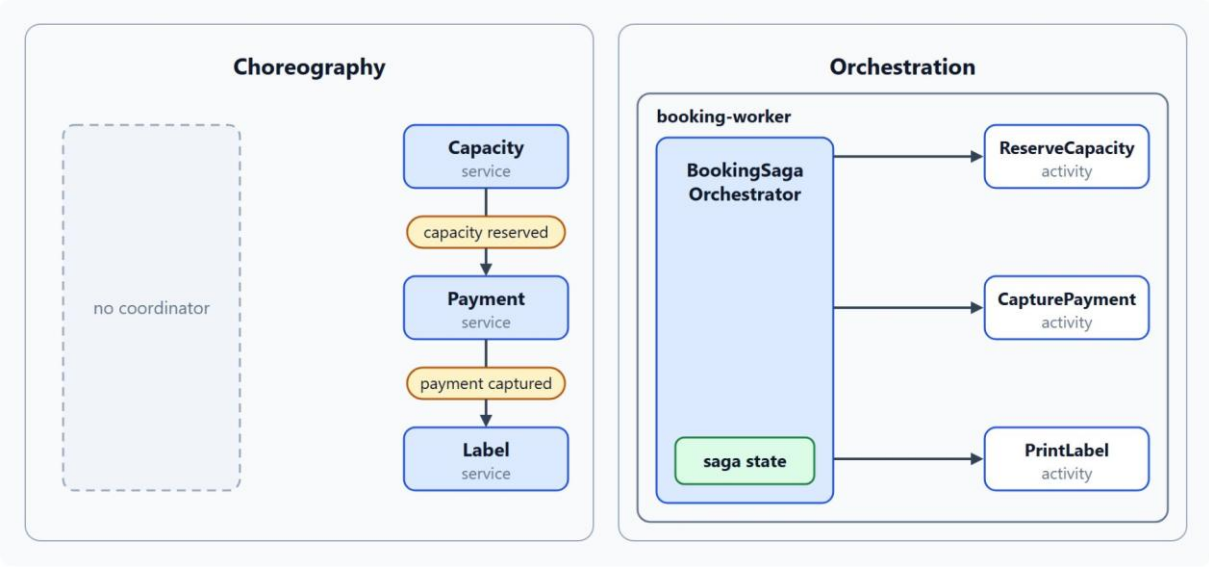
- A **compensable** step can be undone by a transaction with the opposite effect
- The **pivot** is the point of no return, after which compensations no longer apply
- A **retryable** step follows the pivot and must succeed eventually, so it has to be idempotent
- All three booking steps are compensable, so the booking has no pivot
- A message to a carrier would become the pivot, and the steps after it would need retries



Choreography and orchestration

The booking uses one orchestrator for its three services

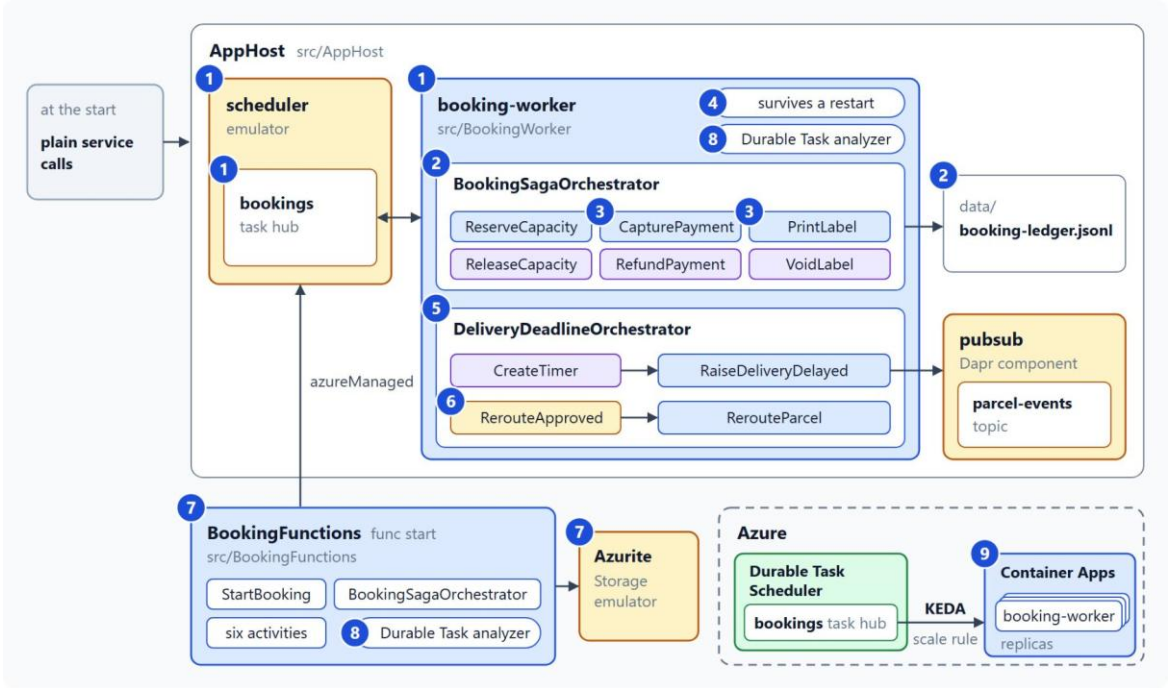
- In **choreography**, each service publishes events and reacts to the events of others
- It suits few services and simple flows, but grows hard to follow and to test end to end
- In **orchestration**, one controller calls each participant and stores the saga state
- It suits complex flows and growing service counts, and needs coordination logic
- The booking uses orchestration; the Module 6 tracking projection uses choreography



Orchestrate the booking saga

What the lab builds in nine steps

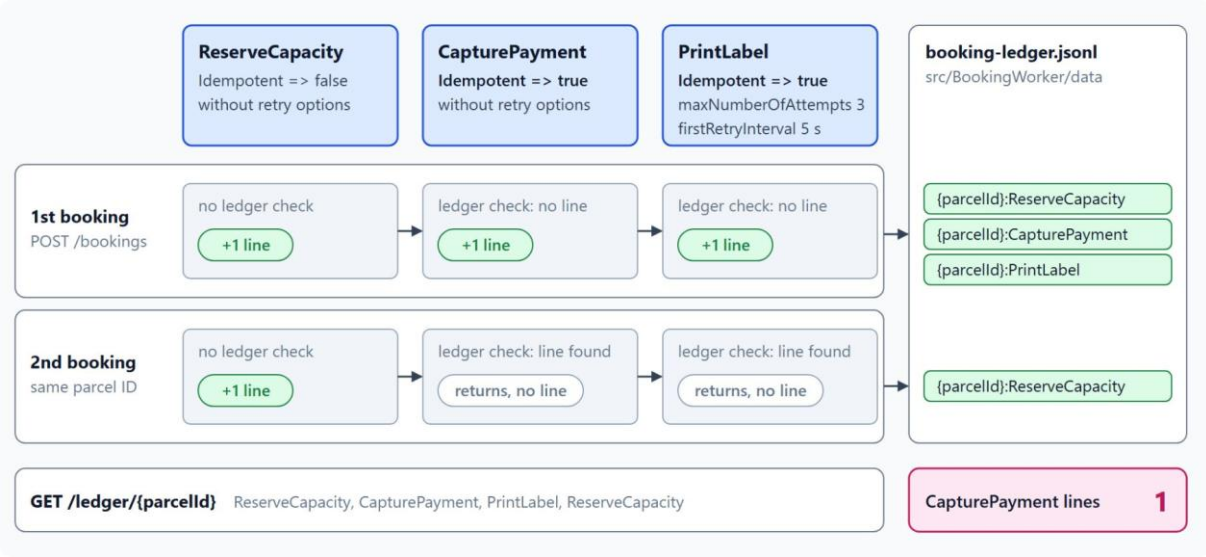
- Steps 1 to 4: the saga on the emulator, its compensations, safe retries and a worker restart
- Steps 5 and 6: a delivery deadline timer and the customer's reroute approval
- Step 7: the same saga as Durable Functions on the `azureManaged` provider
- Step 8: the Durable Task analyzer in both projects
- Step 9: a KEDA scale rule for `booking-worker` on Container Apps



Safe retries and a worker restart

One payment line per parcel, even after a restart

- Only `PrintLabel` gets a retry policy: three attempts, the first retry after 5 seconds
- `CapturePayment` and `PrintLabel` look up the parcel ID plus the activity name in the ledger
- Two bookings with the same parcel ID leave one `CapturePayment` line
- A worker stopped mid-saga hands the instance to the restarted worker, which replays the history
- Parcel IDs with `FAIL-` or `SLOW` make failures and the restart visible



Takeaways

What to carry into the AI agents module

- Keep orchestrators deterministic and put every side effect in an activity
- Give every activity a stable idempotency key, because it runs at least once
- Bound every wait for a person with a durable timer, and cancel the timer that loses
- Scale SDK workers on the scheduler backlog with the KEDA rule
- Undo completed saga steps in reverse order, and alert on compensated sagas

